Efficiency Boost: Service Robot Path Planning with Grey Wolf Optimization

Anel Husaković
Eacon doo, Zenica
Zenica, Bosnia and Herzegovina
anel@eacon.ba

Lejla Banjanović-Mehmedović, Tatjana Konjić
Faculty of Electrical Engineering, University of Tuzla
Tuzla, Bosnia and Herzegovina
lejla.banjanovic-mehmedovic@fet.ba, tatjana.konjic@fet.ba

Abstract—**In the era of Industry 4.0, service robot path planning has emerged as a pivotal element in the optimization of logistic tasks within manufacturing, warehousing and service applications. In this context, the adoption of advanced path planning algorithms, such as the Grey Wolf Optimizer (GWO) swarm algorithm, play a key role in enabling these robots to navigate through complex environments with precision and agility. Harnessing the power of bio-inspired algorithms, our framework establishes a methodical and effective approach to the intricate task of service robot path planning.**

Keywords-path planning, optimization, service robot, great wolf optimizer, Industry 4.0.

I. Introduction

Service robots have become integral components in various domains, including healthcare, hospitality, and personal assistance. These robots are designed to perform tasks that benefit individuals and organizations, ranging from helping patients in hospitals to assisting customers in stores. One of the key innovations in this era is the utilization of service robots to enhance efficiency, accuracy, and flexibility in logistics, too. Service robots are employed in warehouses and distribution centers for tasks such as materials transportation as well as picking, packing, and sorting. Collaborative robots work alongside human workers to enhance productivity in logistics settings in Industry 4.0 and Industry 5.0 [1,2].

Effective path planning is a crucial aspect of service robot design, enabling them to find the most efficient routes within manufacturing facilities, warehouses, and distribution centers as well as to navigate efficiently and interact safely with their surroundings and human users. Cobots can adapt to changing demands and select the quickest paths, ultimately reducing cycle times and increasing throughput. Path planning ensures smooth coexistence of human and robot workers on the factory floor.

The robot path planning problem (RPP) is a classic combinatorial optimization problem [3]. Based on whether the path conditions and environment change over time, path planning can be divided into dynamic and static paths. Path planning can be broken down into global and local paths according to the planning target scope.

In the global path planning robots have complete information about the stationary obstacles and they know in advance the trajectory of the moving obstacles, so they can plan overall paths (offline) in environments before they begin to move. Local path planning has several challenges that include variable environmental conditions, sensor constraints, sensor reading errors, location estimation, robot dynamics, and restrictions on robot motion because it is performed by moving the mobile robot through the environment and using sensors to gather information (online) and update the map and plan its next course of action to create a map of its surroundings [3]. RPP can be applied in industrial, agricultural, military [4] and other fields [5].

The classification of mobile robot path planning algorithms hinges on three primary types distinguished by their features: classical algorithms, artificial intelligence algorithms, and bionic algorithms [6]. The classic solutions are mostly comprised of cell decomposition algorithms (CD), graph search algorithms (Dijkstra algorithm, A* and D* algorithms), Artificial Potential Field algorithm (APF), sampling-based methods (RRT, RRT*, PRM) and dynamic window approach (DWA) [6, 7, 8]. The field is concerned with the issue of high time complexity in high dimensions and the trapping of local minima [3].

Artificial intelligence algorithms incorporate advanced techniques such as neural network algorithms and fuzzy logic systems for effective path planning. Swarm intelligence optimization algorithms, also known as bionic algorithms, draw inspiration from nature and include notable examples such as the genetic algorithm (GA), ant colony algorithm (ACO), particle swarm optimization (PSO), bacterial foraging optimization (BFO), artificial bee colony algorithm (ABC), cuckoo search algorithm (CSA), firefly algorithm (FA), gray wolf algorithm (GWO), and sparrow search algorithm (SSA) [6]. The use of bionic algorithms has increased among researchers to solve path planning problems because they can address issues related to multidimensionality, complex workspaces, and local minima.

The paper [9] presents the path planning of an autonomous mobile disinfection robot based on Simultaneous Localization and Mapping (SLAM), utilizing the Dynamic Window Approach (DWA) and Particle Swarm Optimization (PSO). An improved PSO was used to plan the path of service robots by the authors in [10]. Patle et al. applied FA in mobile robot path planning under uncertain situations [11]. The mobile robot path planning methodology employing the meta-heuristic Grey Wolf Algorithm (GWO) for efficient navigation in modern manufacturing scenarios is presented in

2024 23rd International Symposium INFOTEH-JAHORINA (INFOTEH) | 979-8-3503-2994-0/24/$31.00 ©2024 IEEE | DOI: 10.1109/INFOTEH60418.2024.10495949

979-8-3503-2994-0/24/$31.00 ©2024 IEEE

Authorized licensed use limited to: BMS College of Engineering. Downloaded on November 05,2026 at 09:59:17 UTC from IEEE Xplore. Restrictions apply.

[12]. The modified GWO algorithm (MGWO) is highlighted as the most competent and preferred method for addressing multi-robot route planning challenges [13].

When evaluating the bionics-based path planning algorithms, several key criteria unveil their distinctions. These criteria include parallel search capability, global optimization capability, convergence time, whether the search direction can be adjusted, escape ability of local optimal solution, robustness, and scope of application [6]. Assessing each algorithm against these criteria provides valuable insights into their respective strengths and limitations, aiding in the selection of the most suitable approach for specific applications.

This paper addresses the realm of global static path planning, aiming to determine the most efficient route from an initial point to a predetermined endpoint within a static obstacle-laden road environment using GWO algorithm. The GWO algorithm demonstrated superior performance compared to other deterministic algorithms like Dijkstra, A*, D*, as well as metaheuristic algorithms such as particle swarm optimization (PSO) in terms of both path calculation accuracy and convergence rate [13]. The GWO algorithm demonstrates global optimization capabilities along with a moderate level of robustness, as evidenced in the study [6].

This paper is organized in the following way. In Section 2, the mobile robot path planning in 2D environment is presented. Section 3 presents a mathematical description of the GWO algorithm. Section 4 reports the results of GWO based path planning simulation experiments. Finally, Section 5 summarizes the full text and discusses future research directions.

II. Service Robot Path Planning

Knowing the starting point and end point, avoiding obstacles, and finding the shortest path are three crucial factors. In the 2D environment, the Robot Path Planning (RPP) necessitates a map to locate the starting and destination points. The robot is depicted as a point through a collection of Cartesian coordinate positions (x,y) within the map, which includes numerous static obstacles. The robot operates without prior knowledge of environmental parameters, such as the locations, shapes, and sizes of the obstacles [14]. We are setting some assumptions here:

1) In the environment, there exists a finite set of stationary obstacles, and their heights do not influence moving objects [5].
2) The position and size of obstacles within the motion area remain constant and do not alter over time [5].
3) In the road environment, the motion area is a predefined two-dimensional space bounded by upper and lower limits, which are predetermined. In our paper, we employ bounds of 1 meter in all directions initially. However, this value is subject to modification based on the safety distance from the obstacles, as indicated in the experiment ("Fig. 4").
4) When the estimated robot position ends outside of the bounds, we are setting the estimated position on the bounds.

To define the fitness (evaluation/cost/objective) function we will need two parts: the distance from the robot to the destination and the penalty for fitness function if the robot is on the edge or inside the obstacle.

Authors proposed different cost functions, see [4, 5, 15]. Authors in [15] introduced cost function as sum of distance and fixed scalar multiplier (100) of the violation cost, where violation cost is defined as incremental sum of the mean value of "*a*" parameter of GWO, that is linearly decreased in each iteration.

The distance from robot to the destination and robot to the obstacle will be modeled as follows [14]:

$$F_1(t) = \sqrt{\left(S_x(t) - G_x\right)^2 + \left(S_y(t) - G_y\right)^2} \quad (1)$$

$$D_o(t) = \sqrt{\left(X(t) - O_x\right)^2 + \left(Y(t) - O_y\right)^2} \quad (2)$$

$$F_2(t) = \{\infty, iff\ D_o(t) < \varepsilon\} \quad (3)$$

where t is current iteration, (S_x, S_y) is current position, (G_x, G_y) is goal position, (O_x, O_y) is obstacle position (respectively done for all other obstacles), $(X(t), Y(t))$ - current estimated position.

Total fitness function will be the sum of the two above.

$$F(t) = F_1(t) + F_2(t) \quad (4)$$

Below there will be discussion about using a cost function regarding the decision of taking into account generated points of the population inside or outside of the safety zone that will effect on taking into account the cost function or not ("Fig. 6").

III. Grey Wolf Optimization (GWO)

Grey Wolf Optimization (GWO) is a stochastic algorithm implemented in 2014. by Mirjalili et al [16]. It is part of the class of metaheuristic algorithms used to solve complex optimization problems. GWO has gained recognition in recent years for its efficiency in finding near-optimal solutions for various optimization tasks.

Inspiration for the algorithm is social hierarchy of wolves and hunting mechanism. In the hierarchy of grey wolves, four distinct groups exist: alpha, beta, delta, and omega wolves. The leader or dominant wolf, known as the alpha leads the other wolves in the group. The alpha ($\boldsymbol{\alpha}$) wolf excels as the premier leader in managing the group dynamics among wolves. The beta ($\boldsymbol{\beta}$) wolf holds the second position in the social hierarchy of wolf groups, assisting the alpha leader in various activities. A delta ($\boldsymbol{\delta}$) wolf submits to both alpha and beta wolves, while it exercises authority over omega wolves ($\boldsymbol{\omega}$)[15, 16]. The group hunting strategy represents another captivating social behavior observed in grey wolves. In this strategic approach, the wolves first identify the prey's location and then encircle it under the guidance of the alpha ($\boldsymbol{\alpha}$) wolf.

In the mathematical model concerning the hunting strategy or optimization of grey wolves, it's assumed that the alpha ($\boldsymbol{\alpha}$), beta ($\boldsymbol{\beta}$), and delta ($\boldsymbol{\delta}$) wolves possess superior knowledge regarding the potential location of prey. Consequently, the

Authorized licensed use limited to: BMS College of Engineering. Downloaded on November 05,2026 at 09:59:17 UTC from IEEE Xplore. Restrictions apply.

positions of wolves in the GWO algorithm are updated using the first three best solutions (**α**, **β**, and **δ**). Notably, there are no omega (**ω**) wolves in the GWO code [15, 16]. The mathematical model detailing the hunting mechanism of grey wolves is given below:

$$\vec{D} = \left|\vec{C}.\vec{X}_p(t) - \vec{X}(t)\right| \quad (5)$$

$$\vec{X}(t+1) = \vec{X}_p(t) - \vec{A}.\vec{D} \quad (6)$$

$$\vec{A} = 2\vec{a}.\vec{r}_1 - \vec{a} \quad (7)$$

$$\vec{C} = 2.\vec{r}_2 \quad (8)$$

$$a = 2 - \frac{2i}{maxIteration} \quad (9)$$

where t denotes the current iteration, $\vec{A}$ and $\vec{C}$ represent coefficient, $\vec{X}_p$ signifies the position vector of the prey, $\vec{X}$ denotes the position vector of a grey wolf, dot product (.) represents scalar product, $\vec{X}(t+1)$ indicates the estimated position in the next iteration, a symbolizes the component linearly decreased from 2 to 0 over the iterations, $\vec{r}_1$ and $\vec{r}_2$ are random vectors ranging between [0,1] [16]. Since $\vec{X}_p$ is treated as the best solution for i-th iteration we can safely change $\vec{X}_p$ to **α**, **β**, and **δ** as the prey to other grey wolves at i-th iteration, so the equations can be written as [15]:

$$\vec{D}_\alpha = \left|\vec{C}.\vec{X}_\alpha(t) - \vec{X}(t)\right| \quad (10)$$

$$\vec{D}_\beta = \left|\vec{C}.\vec{X}_\beta(t) - \vec{X}(t)\right| \quad (11)$$

$$\vec{D}_\delta = \left|\vec{C}.\vec{X}_\delta(t) - \vec{X}(t)\right| \quad (12)$$

$$\vec{X}_\alpha(t)^* = \vec{X}_\alpha(t) - \vec{A}.\vec{D} \quad (13)$$

$$\vec{X}_\beta(t)^* = \vec{X}_\beta(t) - \vec{A}.\vec{D} \quad (14)$$

$$\vec{X}_\delta(t)^* = \vec{X}_\delta(t) - \vec{A}.\vec{D} \quad (15)$$

$$\vec{X}(t+1) = (\vec{X}_\alpha(t)^* + \vec{X}_\beta(t)^* + \vec{X}_\beta(t)^*)/3 \quad (16)$$

Each iteration, search agent updates its position according to **α**, **β**, and **δ** in a 2D search space. The final position will be located randomly within a circle determined by the positions of the alpha (**α**), beta (**β**), and delta (**δ**) wolves in the search space. Here, the alpha, beta, and delta wolves estimate the position of the prey, while the remaining wolves update their positions randomly around the prey [16].

IV. Results of GWO in Avoiding Collisions with Obstacles

A. GWO implementation

In the pseudo code, "Fig. 1", one can see that for the cost function of interest, it is noted that without changing the range of movement in the algorithm, algorithm traps in the local minima for specified environment, which will be shown in diagrams below. This is in line with the defect of the algorithm found in [17], where the algorithm tries to solve the cost function far away from the optimal solution. Solution we found is to expand the boundary range by 1% increment in case there is no difference between score values of best agents between two successive iterations. Using this approach, we managed to move from local optimums.

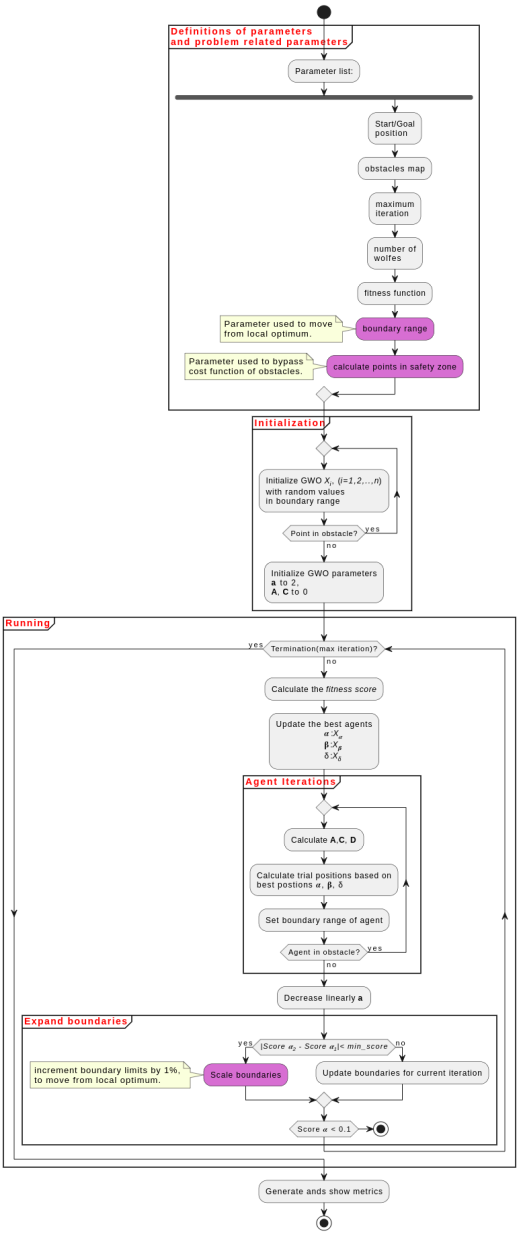


Figure 1. Pseudo code used in GWO implementation.

Authorized licensed use limited to: BMS College of Engineering. Downloaded on November 05,2026 at 09:59:17 UTC from IEEE Xplore. Restrictions apply.

B. Simulation results experiments

The following experiments were done by changing the maximum number of iterations only. Results are obtained with Matlab R2023a with Navigation Toolbox, on AMD Ryzen 7 2700X 8-Core Processor, 32GB RAM, GeForce GTX 1060 6GB.

As variable parameters we used GWO algorithm specific parameters like the number of iterations (N), number of wolves population (W), boundary range (r) and cost function related parameter epsilon (ε), where ε is the positive number that acts as the safety zone from obstacles. For each test we measured metrics, namely time to converge, clearance and path smoothness.

"Fig. 2." (1.row) shows the best shortest path and score function for N= 300 and fixed parameters (ε= 1[m], r= 1[m], W= 10), as well as all other tested paths and scores (2. row) based on the variable number of iterations (from 50 to 300). It takes about 2.1[s] to converge.

We measured impact of changing the wolf population (W) and increase in this number (from 30-100) led to slower and not optimal solutions, while decreasing the number to 5 converged faster (about 1.8 [s]), but since we used as a measure shortest path/smoothness, it was worse than previous example for W=10.

The boundary range is the parameter whose value is equal to half of the square length with the center in the alpha (best) position. The parameter is used in order to derail from local optimum as well as to get the smoother paths with effect shown in "Fig. 3".

To jump from local optimum, we are suggesting (highlighted in "Fig. 1"), a small tweak of the algorithm by scaling the square range by positive scalar factor in a some direction, whenever the difference of the sequential scores is less than predefined parameters. By experiments it is concluded that a scalar factor of 1[%] is good enough for scaling the boundaries and the difference of sequential scores must be less than 0.01. The limitations of this approach are to determine in which directions to scale and cover all directions.

"Fig. 4." (1. row) shows the best shortest path and score function for fixed parameters (ε= 1[m], W= 10, N= 300) and r=1.2 [m] as well as all other tested paths and scores (2. row) based on the variable boundary range r (from 0.1[m]-2[m]). It takes around 2.3[s] to converge and leads to a smoother path compared to "Fig. 2". r= 1[m] is used as the optimal parameter since there was no major difference in metrics.

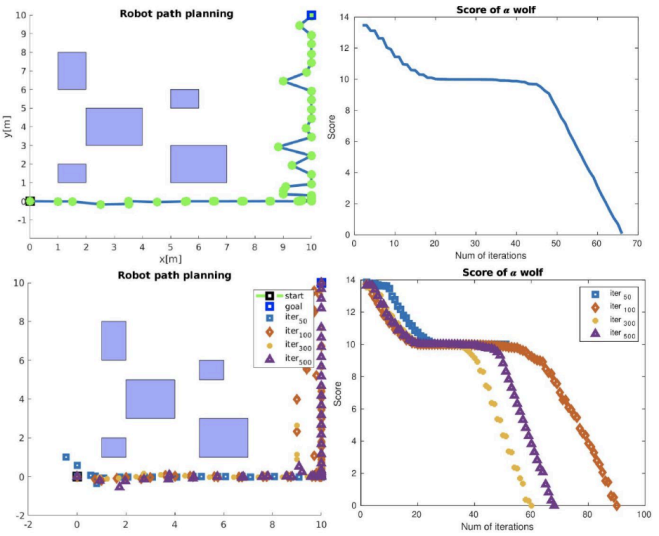


Figure 2. Shortest path and score function for the optimal number of iterations (1. row with parameters ε= 1[m], r= 1[m], W= 10, N=300) and path/scores for other experiments (2. row).

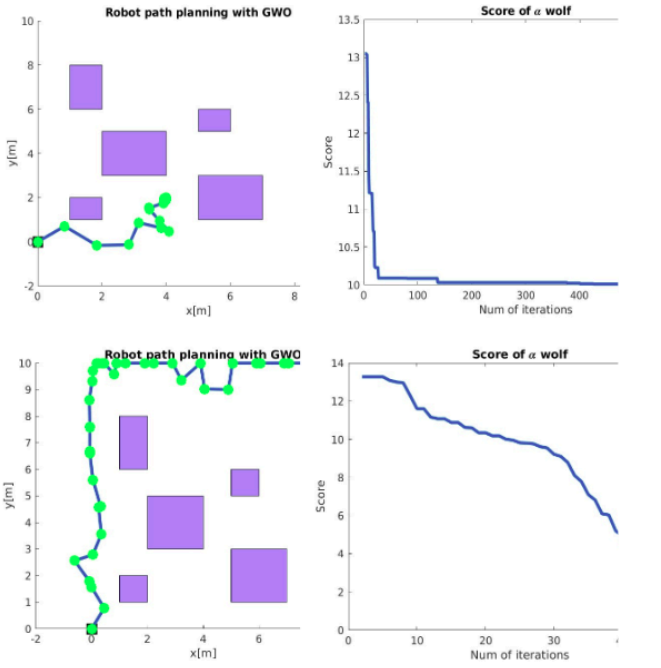


Figure 3. Path/scores without boundary range parameter r (up) and with r (down).

Authorized licensed use limited to: BMS College of Engineering. Downloaded on November 05,2026 at 09:59:17 UTC from IEEE Xplore. Restrictions apply.

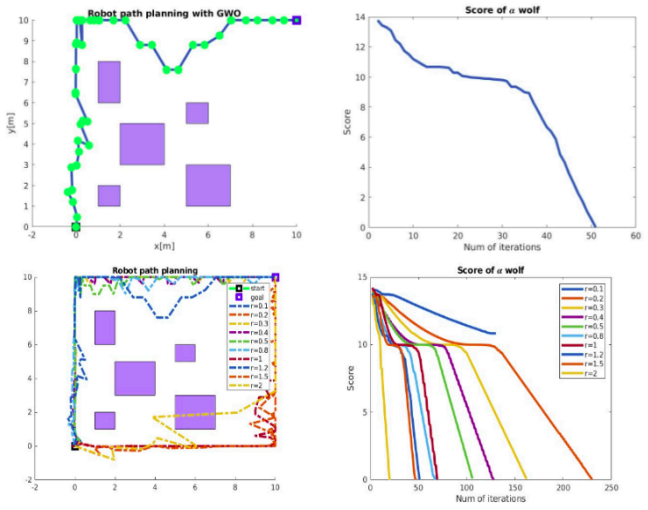


Figure 4. Shortest path and score function for the optimal value of boundary range (1. row, ε= 1[m], r= 1[m], W= 10, N=300) and path/scores for other experiments (2. row).

The experiments showed that there is no major difference in approach of taking the cost function of obstacles, with points instantiated inside the safety zone ε ("Fig. 5") and preventing creation of cloud data in the safety zone, in which case cost function for the obstacles inside the safety zone does not exist ("Fig. 6"). We used a later approach.

"Fig. 7" (1. row) shows the best shortest path and score function for fixed parameters (r= 1[m], W=10, N=300) and ε=0.5 [m] as well as all other tested paths and scores (2. row) based on the variable safety zone ε (from 0.1[m] - 2[m]; ε is represented as "r" in figure). It takes about 1.7[s] to converge and leads to a smoother path compared to "Fig. 4".

With optimal parameters ε= 0.5 [m], r= 1[m], W= 10 and N= 300, we changed to goal positions and measured the metrics. "Fig. 8" shows the changed goal position from point (10, 10) to point (4, 6). Algorithm finished after 55 iterations in 2.4 [s], respecting the clearance of 0.5 [m].

The similar was done for the next goal position shown in the "Fig. 9", which shows the changed goal position at point (3, 8), with similar metric as in previous example.

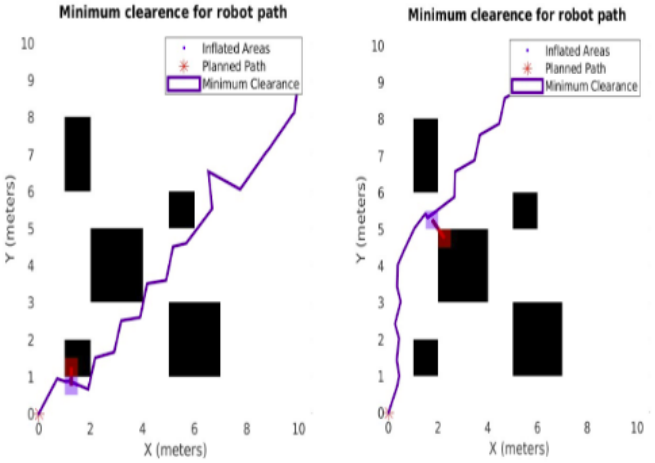


Figure 5. Shortest path and minimal clearance for ε=0.1[m] (left) and ε=0.5[m] (right).

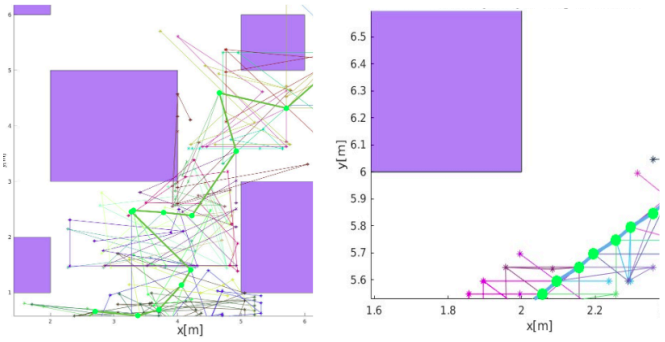


Figure 6. Paths that are taking into account points in the safety zone (left) and paths that are not taking into account points in the safety zone (right), for different *ε (0.3 [m])* and boundary movement range (*0.1 [m])*.

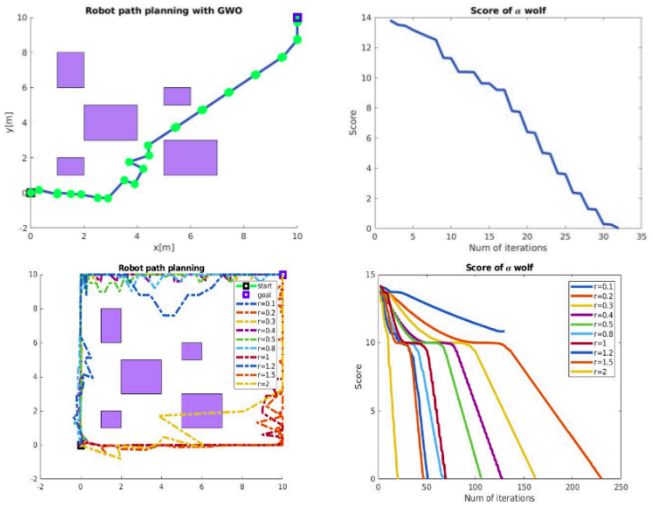


Figure 7. Shortest path and score function for the optimal safety zone ε (1. row ε= 0.5[m], r= 1[m], W= 10, N=300) and path/scores for other experiments (2. row).

Authorized licensed use limited to: BMS College of Engineering. Downloaded on November 05,2026 at 09:59:17 UTC from IEEE Xplore. Restrictions apply.

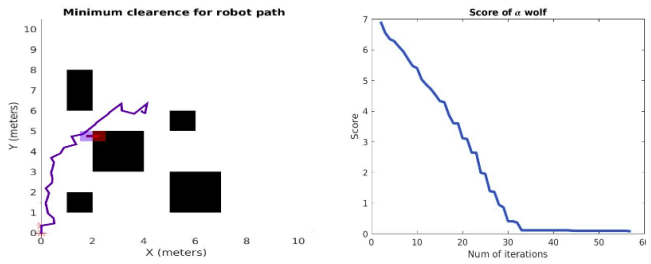


Figure 8. Changing the goal position to (4,6) with optimal parameters ε= 0.5[m], r= 1[m], W= 10, N=300.

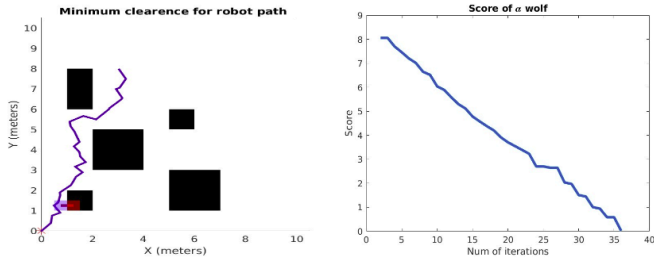


Figure 9. Changing the goal position to (3,8) with optimal parameters ε= 0.5[m], r= 1[m], W= 10, N=300.

V. Conclusion

As Industry 4.0 continues to evolve, path planning algorithms will play an increasingly central role in providing that mobile robots contribute to enhanced productivity, safety, and sustainability in the industrial landscape. The optimization of mobile robot paths in Industry 4.0 applications, particularly through bionic optimization algorithms, represents a promising avenue for enhancing efficiency and safety, within complex environments, ultimately contributing to increased productivity and cost savings.

GWO algorithm is quite well performing in terms of safety, shortest path and resource/energy efficiency. Paper showed how to properly tune parameters like maximum number of iterations, population size, range function versus safety zone in order to obtain desired results. For the selected cost function and the environment modeling we noted the importance of boundary constraints for which the algorithm remains in local optima, that is in line with research done in [15].

We proposed incremental increased range when updating the positions for the next agents for each iteration, whose impact is most visible at first iterations where cost functions are highest. This eliminates local minima and proves more stability of the algorithm.

Future research directions may be oriented toward the creation of an innovative mobile robot path planning strategy based on modified bionic algorithms, for both static and dynamic environments, with a specific focus on fostering seamless multi-robot and human-robot collaboration.

References

[1] T. Arai, T. Arai, and T. Arai, "Intelligent Robotics and Applications." Springer, 2016.

[2] M. K. Habib, et al., "Autonomous Robots in Logistics: A Comprehensive Review." Robotics, 8(1), 14, 2019.

[3] J. A. Abdulsaheb and D. J. Kadhim, "Classical and Heuristic Approaches for Mobile Robot Path Planning: A Survey", Robotics 2023, 12(4), 93, 2023.

[4] F. Kiani, A. Seyyedabbasi, S. Nematzadaeh, F. Candan, T. Cevik, F. A. Anka, G. Randazzo, S. Lanza and A. Muzirafuti, "Adaptive Metaheuristic-Based Methods for Autonomous Robot Path Planning: Sustainable Agricultural Applications", Applied Sciences 12(3):943, 2022.

[5] C. Wang, S. Cheng, Q. Zhao and Y. Suo, "An Efficient End-to-End Obstacle Avoidance Path Planning Algorithm for Intelligent Vehicles Based on Improved Whale Optimization Algorithm", Mathematics 2023, 11(8), 1800, 2023.

[6] L. Liu, X. Wang, X. Yang, H. Liu, J. Li, P. Wang: "Path planning techniques for mobile robots: Review and prospect", Elsevier Expert Systems With Applications, 2023.

[7] Y. Tian, X. Li, H. Ma and X.Zhang, "Deep Reinforcement Learning Based Adaptive Operator Selection for Evolutionary Multi-Objective Optimization", IEEE TRANSACTIONS ON EMERGING TOPICS IN COMPUTATIONAL INTELLIGENCE, VOL. 7, NO. 4, 2023.

[8] L. Liu, X. Wang, X. Yang, H. Liu, J. Li and P. Wang, "Path planning techniques for mobile robots: Review and prospect", Expert Systems With Applications Vol. 227, 2023.

[9] L. Banjanovic-Mehmedovic, I. Karabegovic, J. Jahic and M. Omercic, "Optimal path planning of a disinfection mobile robot against COVID-19 in a ROS-based research platform", Advances in Production Engineering & Management, Volume 16, Number 4, 2021, pp 405-417.

[10] X. Zhan and Z. Chen, "Path Planning of Service Robot Based on Improved Particle Swarm Optimization Algorithm," 2023 4th International Symposium on Computer Engineering and Intelligent Communications (ISCEIC), Nanjing, China, 2023, pp. 244-248, doi: 10.1109/ISCEIC59030.2023.10271235.

[11] B.K. Patle, A. Pandey, A. Jagadeesh, et al.: "Path planning in uncertain environment by using firefly algorithm", Defence technology, 14(6), 691–701, 2018.

[12] A.Pawlowski, S.Romaniuk, Z.Kulesza and M.Petrovic, "Trajectory optimization using learning from demonstration with meta-heuristic grey wolf algorithm", IAES International Journal of Robotics and Automation (IJRA), Vol. 11, No. 4, pp. 263~277, 2022.

[13] R. Kumara, L. Singhb, and R. Tiwaric: "Path planning for the autonomous robots using modified grey wolf optimization approach", Journal of Intelligent & Fuzzy Systems 40 9453–9470, 2021.

[14] F. A. Tahseen and H. S. Alaa, "Safe and Optimum Navigation of Wheeled Mobile Robot using Grey Wolf Optimization Algorithm", IOP Conference Series Materials Science and Engineering, 928(2), 2020.

[15] D. Lokman and Y. Ugur, "Robot Path Planning using Gray Wolf Optimizer", International Conference on Advanced Technologies, Computer Engineering and Science (ICATCES18), 2018.

[16] S. Mirjalili, S. M. Mirjalili and A. Lewis, "Grey Wolf Optimizer. Advances in Engineering Software", Advances in Engineering Software Volume 69, 46:6, 2014.

[17] P. Niu, S. Niu, L.Nan, L. Chang: "The defect of the Grey Wolf optimization algorithm and its verification method", Knowledge-Based Systems Volume 171, 37:43, 2019.

Authorized licensed use limited to: BMS College of Engineering. Downloaded on November 05,2026 at 09:59:17 UTC from IEEE Xplore. Restrictions apply.